Nom:	Prénom:	Note:

Exercice 1 ►

Cet exercice porte sur l'architecture matérielle (réseau), les arbres binaires et la programmation Python.

L'entreprise CaféNet possède plusieurs cafés répartis dans différentes villes. Le réseau de la chaîne de cafés est représenté en Figure 1.

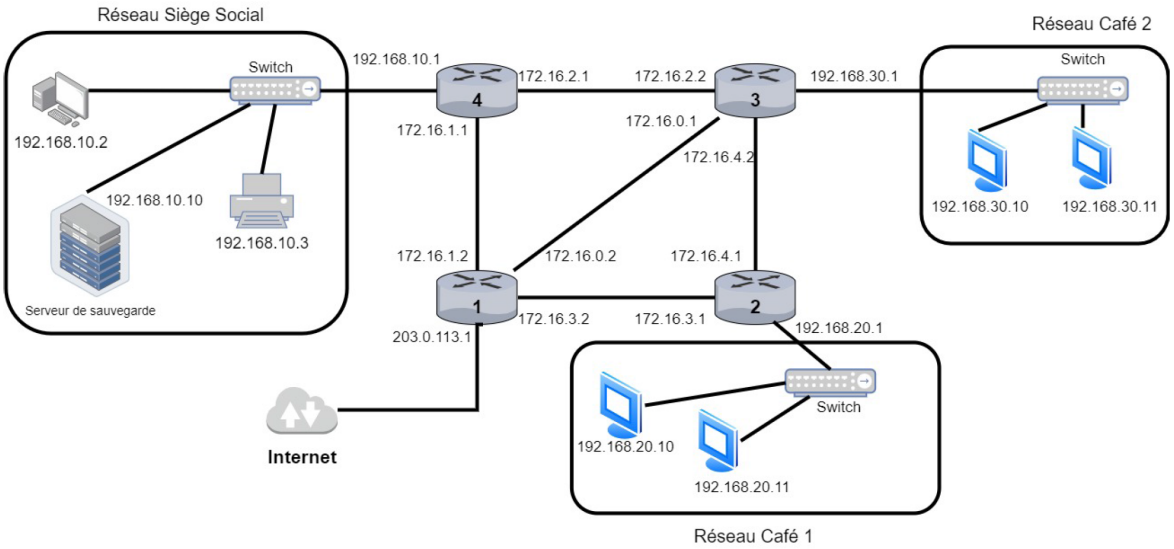

Figure 1. Schéma d'une partie du réseau

Sur le schéma sont représentés 4 routeurs, le réseau du siège social, le réseau du café 1, le réseau du café 2. Dans les réseaux du café 1 et du café 2, des bornes de commandes sont connectées à des switchs (ce sont des boîtiers de connexion qui n'ont pas eux-mêmes d'adresse IP). Les 4 routeurs représentés sont composés d'au moins 3 interfaces réseau capable de relier des réseaux ensemble. Chaque interface possède donc une adresse IPv4 sur le réseau auquel elle est reliée.

Les masques des sous-réseaux sont tous 255.255.255.0. Avec ce masque, les trois premiers octets des adresses IP codent l'adresse réseau. Le dernier octet, c'est-à-dire les 8 derniers bits, code l'adresse des machines à l'intérieur de chaque sous-réseau.

PARTIE A

1. Indiquer l'adresse réseau du réseau du siège social.

   - le schéma montre une adresse `192.168.10.2` avec un masque de sous réseau `255.255.255.0`.
   - avec ce genre de masque, les trois premiers octets codent la partie réseau de l'adresse IP (NetId)
   - les trois premiers octets sont donc `192.168.10`
   - on complète avec des `0`.
   - l'adresse réseau du siège social est donc `192.168.10.0`

2. Indiquer l'adresse réseau du réseau du café 1.

   De la même manière : `192.168.20.0`

Le gérant veut faire installer une troisième borne de commande dans le café 1.

3. Indiquer les deux seules adresses IP valides pour cette nouvelle borne, parmi les quatre adresses IP proposées. Entourer les bonnes réponses.

`192.168.20.2`	`192.168.20.157`	`192.168.20.261`	`192.168.24.12`

> Les adresses IP valides sont `192.168.20.2` et `192.168.20.157`. En effet, l'adresse `192.168.20.261` n'est pas valide car le dernier octet doit être compris entre 0 et 255. L'adresse `192.168.24.12` n'est pas valide car elle n'appartient pas au réseau du café 1.

L'adresse de diffusion, appelée aussi adresse de Broadcast, est la dernière adresse disponible à l'intérieur d'un réseau local.

4. Déterminer l'adresse de diffusion du réseau du café 1.

> `192.168.20.255`

5. Déterminer combien de machines informatiques il est encore possible de connecter au réseau du café 1 après l'installation de la troisième borne de commande.

> $2^8 - 2 - 3 = 256 - 2 - 3 = 251$ machines.

Le réseau local du café 1 n'a pas besoin de plus de 8 adresses IP différentes. Ce décompte d'adresses IP inclut les adresses IP réservées (à savoir l'adresse de diffusion et l'adresse du réseau). Il est rappelé que la longueur du masque de sous-réseau est actuellement de 24 bits (c'est-à-dire 3 octets).

6. Proposer un nouveau masque de sous-réseau permettant d'avoir au maximum 8 adresses IP différentes dans le réseau du café 1 (6 adresses utilisables). Justifier votre réponse.

> Avec un masque de sous-réseau de 24 bits, il y a 8 bits pour coder les adresses des machines à l'intérieur du réseau du café 1. Cela permet d'avoir $2^8 = 256$ adresses différentes, dont 254 utilisables (en retirant l'adresse de diffusion et l'adresse du réseau). Pour n'avoir que 8 adresses différentes, il faut réduire le nombre de bits disponibles pour coder les adresses des machines. En utilisant un masque de sous-réseau de 29 bits, il ne reste que 3 bits pour coder les adresses des machines, ce qui permet d'avoir $2^3 = 8$ adresses différentes, dont 6 utilisables (en retirant l'adresse de diffusion et l'adresse du réseau). Le nouveau masque de sous-réseau est donc `255.255.255.248`.

PARTIE B

RIP (Routing Information Protocol) est un protocole de routage utilisé dans les réseaux IP. Il est conçu pour réduire le nombre de sauts entre deux réseaux. Un « saut » correspond au transfert des données d'un routeur à un autre. Le protocole RIP utilise le nombre de sauts comme critère principal pour évaluer le coût d'un chemin. Autrement dit, il considère que le chemin le plus optimal est celui qui traverse le moins de routeurs.

La table de routage du routeur 2 de la Figure 1 est représentée ci-dessous :

Réseau destination	**Interface de sortie**	**Prochain routeur**	**Nombre de sauts**
`192.168.20.0`	`192.168.20.1`	aucun	0
`172.16.3.0`	`172.16.3.1`	aucun	0
`172.16.4.0`	`172.16.4.1`	aucun	0
`192.168.10.0`	`172.16.3.1`	`172.16.3.2`	2
`172.16.0.0`	`172.16.4.1`	172.16.4.2	1
`172.16.2.0`	`172.16.4.1`	172.16.4.2	1
`192.168.30.0`	`172.16.4.1`	172.16.4.2	1
`172.16.1.0`	`172.16.3.1`	172.16.3.2	1

7. Compléter les cases vides de la table de routage du routeur 2 ci-dessus.

Une adresse IP qui n'est pas référencée dans la table de routage doit être routée par défaut vers Internet.

8. Compléter la ligne à ajouter à la table de routage du routeur 2.

Réseau destination	Interface de sortie	Prochain routeur	Nombre de sauts
autre	172.16.3.1	172.16.3.2	1

La connexion entre le routeur 2 et le routeur 3 est interrompue. Le protocole RIP va permettre au routeur 2 de trouver un nouveau chemin pour atteindre les réseaux du café 2.

9. Indiquer la route utilisée par le routeur 2 pour atteindre les réseaux du café 2 après l'interruption de la connexion entre le routeur 2 et le routeur 3.

R2 - R1 - R3

10. Compléter alors la nouvelle table de routage du routeur 2 après l'interruption de la connexion.

Réseau destination	Interface de sortie	Prochain routeur	Nombre de sauts
192.168.30.0	172.16.3.1	172.16.3.2	2

PARTIE C

OSPF est également un protocole d'échanges de données entre les routeurs qui prend en compte le coût des routes. Le coût est lié au débit des liaisons entre les routeurs par la formule suivante :

$$cout = \frac{10^9}{Debit\ de\ la\ liaison\ en\ bits/s}$$

11. Compléter la dernière colonne du tableau ci-dessous :

Type de connexion	Débit en bit/s	coût
Ethernet	10 Mbit/s	100
Fast Ethernet	100 Mbit/s	10
Fibre Optique	1 Gbit/s	1

Le schéma ci-dessous met en évidence les types de connexion qui relient les routeurs.

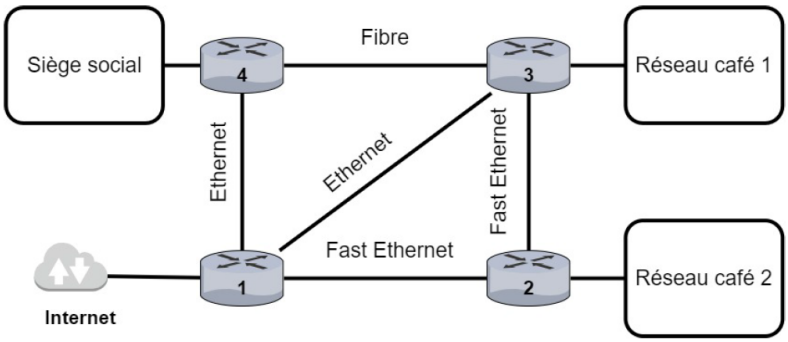

Figure 2. Schéma des types de connexion

12. Compléter le schéma de la Figure 2 en indiquant les coûts des différentes connexions entre les routeurs.
13. Compléter alors la table de routage du routeur 2 en utilisant le protocole OSPF.

Réseau destination	Prochain routeur	Coût
Siège social	R3	11
Réseau café 1	R3	10
Réseau café 2	-	0
Internet	R1	10

14. Déterminer la route dont le coût est minimal pour aller d'Internet jusqu'au siège social. Calculer son coût au sens du protocole OSPF.

R1 - R2 - R3 - R4 de cout 21

PARTIE D

Le but de cette partie est de classer les adresses IP des différents réseaux afin de faciliter leur recherche.

La fonction `ip_bin` prend en argument une chaîne de caractères décrivant une adresse IP en notation décimale, et renvoie une chaîne de caractères, de longueur 35 (32 bits et les 3 points), décrivant l'adresse IP en notation binaire.

Exemple :

Code Python
```python
>>> ip_bin('192.168.10.1')
'11000000101010000000101000000001'
```

15. Donner la chaine de caractères renvoyée par `ip_bin('192.168.20.12')`.

'11000000101010000001010000001100'

La fonction `precede` prend en paramètres deux adresses IP en notation binaire, sous forme de chaînes de caractères identiques à celles renvoyées par la fonction `ip_bin`. La fonction `precede` renvoie un booléen qui vaut `True` si la première adresse IP en paramètre précède la seconde adresse IP.

Exemple :

Code Python
```python
>>> a = '11000000101010000000101000000001'
>>> b = '11000000101010000000111100000001'
>>> precede(a, b)
True
```

L'algorithme compare bit à bit les deux chaînes binaires, en lisant les chaînes de caractères dans le sens usuel (de gauche à droite). Dans l'exemple ci-dessus, tous les caractères sont identiques jusqu'au sixième caractère du troisième octet. Comme le bit de l'adresse a est inférieur à celui de l'adresse b, on en déduit que l'adresse IP a précède l'adresse IP b.

Si la première adresse IP ne précède pas la seconde, la fonction doit renvoyer `False`.

L'algorithme de comparaison est traduit dans le langage Python sous la forme suivante :

Code Python

```python
1 def precede(ip_1, ip_2):
2     if ip_1 == '':
3         return False
4     if ip_1[0] < ip_2[0]:
5         return ...
6     if ip_1[0] > ip_2[0]:
7         return ...
8     return precede(... , ...)
```

16. Compléter les lignes 5, 7 et 8 de la fonction `precede`

Code Python

```python
def precede(ip_1, ip_2):
    if ip_1 == '':
        return False
    if ip_1[0] < ip_2[0]:
        return True                # ← Ligne 5
    if ip_1[0] > ip_2[0]:
        return False               # ← Ligne 7
    return precede(ip_1[1:], ip_2[1:])  # ← Ligne 8
```

17. Dans le cas où les deux adresses IP sont identiques, que renvoie la fonction `precede`

La fonction `precede` renvoie `False` dans ce cas.

PARTIE E

L'objectif de cette partie est de :

- prédire si deux machines appartiennent au même réseau local,
- renvoyer l'adresse du réseau local auquel appartient une machine,

Chaque machine est caractérisée par une adresse IP en notation CIDR.

Code Python

```python
PC0 = {'nom': 'PC0', 'adresse_IP': '192.168.4.5/24'}
PC1 = {'nom': 'PC1', 'adresse_IP': '192.168.4.145/24'}
```

Code Python

```python
def ip_bin(adresse):
    ''' Convertit une adresse Ip en une adresse IP en notation binaire. '''
    # Le code n'est pas demandé

assert ip_bin('192.168.4.5') == '11000000101010000000010000000101'

def masque_SR(nb_uns):
    ''' Renvoie le masque de sous-réseau correspondant
    à une longueur de masque donnée en nombre de bits à 1. '''
    # Le code n'est pas demandé

assert masque_SR(24) == '255.255.255.0'

def extraire_IP(adresse):
    ''' Extrait l'adresse IP d'une adresse en notation CIDR. '''
    # Le code n'est pas demandé

assert extraire_IP('192.168.4.5/24') == '192.168.4.5'

def extraire_MQ(adresse):
    ''' Extrait le masque de sous-réseau d'une adresse en notation CIDR. '''
    # Le code n'est pas demandé

assert extraire_MQ('192.168.4.5/24') == '24'
```

18. En utilisant les fonction précédentes, compléter la fonction `adresse_reseau` qui renvoie l'adresse du réseau local auquel appartient une machine.

```python
def adresse_reseau(machine):
    ''' Renvoie l'adresse du réseau local auquel appartient une machine '''

    # Extraire les composants
    ip = extraire_IP(machine)
    nb_bits = int(extraire_MQ(machine))

    # Convertir IP en binaire
    ip_binaire = ip_bin(ip)

    # Créer le masque
    masque = masque_SR(nb_bits)
    masque_binaire = ip_bin(masque)

    # Appliquer le masque (AND logique)
    reseau_binaire = ''
    for i in range(len(ip_binaire)):
        if masque_binaire[i] == '1':
            reseau_binaire += ip_binaire[i]
        else:
            reseau_binaire += '0'

    # Convertir le résultat binaire en IP décimale
    # (appel une fonction inverse de ip_bin, non fournie)
    return convertir_binaire_en_ip(reseau_binaire)
```

```
**Ou version plus concise** :

def adresse_reseau(machine):
    ''' Renvoie l'adresse du réseau local auquel appartient une machine '''
    ip = extraire_IP(machine)
    nb_bits = int(extraire_MQ(machine))
    masque = masque_SR(nb_bits)

    # Appliquer le masque en binaire
    ip_bin_result = ip_bin(ip)
    masque_bin = ip_bin(masque)

    reseau_bin = ''.join(
        ip_bin_result[i] if masque_bin[i] == '1' else '0'
        for i in range(len(ip_bin_result))
    )

    return convertir_binaire_en_ip(reseau_bin)

assert adresse_reseau('192.168.1.4/24') == '192.168.1.0'
assert adresse_reseau('172.18.1.4/16') == '172.18.0.0'
```

19. En utilisant les fonctions précédentes, compléter la fonction `meme_reseau` qui renvoie `True` si les deux machines appartiennent au même réseau et `False` sinon.

```
def meme_reseau(machine1, machine2):
    ''' Renvoie True si les deux machines appartiennent au même réseau
    et False sinon. '''
    return adresse_reseau(machine1) == adresse_reseau(machine2)
assert meme_reseau('192.168.1.4/24', '192.168.1.15/24') == True
assert meme_reseau('192.168.1.4/24', '192.168.2.5/24') == False
assert meme_reseau('192.168.1.4/16', '192.168.2.5/16') == True
```